

ROCK implementation in Python with Code by Victor Rielly

Adam Handwerger

Recount ROCK Objective

We seek an f which minimizes $J(f) =$

$$\left\{ \sup_{v(t): \|v(t)\|=1} \left[\left(\int_a^b (\dot{x}(t) - f(x(t)))^\top v(t) dt \right)^2 \right] + \lambda \|f\|^2 \right\} \quad (1)$$

where $f \in$ an vvRKHS and $v(t)$ is d-dimensional test function from a finite dimensional vvRKHS, V with a p-dimensional feature map, ϕ_τ .

Using ROCK to reframe problem

Using the the theory of ROCK, (1) can be reframed as solving the following linear system for α :

$$(M + \lambda I) \alpha = y, \text{ where } y \in \mathbb{R}^{npd} \quad (2)$$

Thus we will need to construct the matrix M and the vector y , but first we will define the block diagonal matrix $\Psi(t)$.

The Test Function Matrix $\Psi(t)$

If p is the dimension of the feature map ϕ_τ for V , and m_i is the number of samples of each trajectory then for each trajectory, there is a corresponding block in Ψ given by

$$\psi_{i,i} = [\psi_i(t_1^i), \dots, \psi_i(t_{m_i}^i)] \in \mathbb{R}^{p \times m_i}$$

Code for Constructing ψ_i using Legendre Polynomials as Components of the Test Functions

For this example the vector-valued test function $v(t)$ for trajectory i has components which are given by the scaled/shifted Legendre Polynomials defined on time interval $[a_i, b_i]$. Each component is found recursively up to order p using the well known recursion formula for Legendre Polynomials. The following code calculates $\psi_i(t^i)$ and its time derivative, $\dot{\psi}_i(t^i)$ for each element in the time array with $t^i \in [a_i, b_i]$. Note that $P_1(t) \propto t$ so that the recursion can start on each element of the time array.

```

# We will be using Scaled and Shifted Legendre Polynomials
# We need the derivatives as well.
# ts - The array of times to evaluate the Legendre polynomials at
# n - The order of the polynomial (integer with 0 <= n).
# a - The left end of the interval to shift the polynomials to
# b - the right end of the interval to shift the polynomials to. (b >
      a)
# Computes the derivative of the Legendre Polynomials based on the
      recursion:
#  $P_{n+1}(x) = [(2n+1)xP_n(x) - nP_{n-1}(x)]/(n+1)$ , which implies:
#  $P'_{n+1}(x) = [(2n+1)P_n(x) + (2n+1)xP'_n(x) - nP'_{n-1}(x)]/(n+1)$ 
def phisp(ts,n,a,b):

    # Apply the transformation of ts to tsp (on the interval [-1,1])
    tsp = (2/(b-a))*ts - (b+a)/(b-a)
    if n == 0:
        return np.ones((len(ts),1)), np.zeros((len(ts),1))
    Ts = np.ones((len(ts),n+1))
    Tsp = np.zeros((len(ts),n+1))
    Ts[:,1] = tsp
    Tsp[:,1] = 1
    for i in range(2,n+1):
        Ts[:,i] = ((2*(i-1)+1)*tsp*Ts[:,i-1] - (i-1)*Ts[:,i-2])/(i)
        Tsp[:,i] = ((2*(i-1)+1)*Ts[:,i-1] + (2*(i-1)+1)*tsp*Tsp[:,i-1] - (i-1)*
                    Tsp[:,i-2])/(i)
    # The normalization term for the scaled Legendre Polynomial is
    #  $\sqrt{(2*n+1)/2} * \sqrt{2/(b-a)}$ 
    Ts = Ts*np.sqrt((np.array(range(n+1))*2+1)/2)[None,:]*np.sqrt(2/(b-a))
    Tsp = Tsp*np.sqrt((np.array(range(n+1))*2+1)/2)[None,:]*np.sqrt(2/(b-a))

    return Ts,Tsp

```

The Vector $y \in \mathbb{R}^{npd}$

To construct the vector y we stack the n vectors y_i given by

$$y_i = \int_{a^i}^{b^i} \Psi_i(t) \dot{x}^i(t) dt$$

which themselves are stacked vectors. For each i there are p vectors y_{ij} which are given by

$$y_{ij} = x_i(b_i)[\psi(b_i)]_j - x_i(a_i)[\psi(a_i)]_j - \int_{a^i}^{b^i} x_i(t) [\dot{\psi}_i(t)]_j dt \in \mathbb{R}^d$$

where $j \in \{1, 2, \dots, p\}$

Construction of y Using Matrix Multiplication

We first construct two related matrices $QPhi$ and $QPhiD$.

Both matrices are block diagonal and are contained in $\mathbb{R}^{pn \times (\sum_i^n m_i)}$, with each block given by

$$QPhi_{i,i} = [q_1^i \Psi_i(t_1^i), \dots, q_{m_i}^i \Psi_i(t_{m_i}^i)]$$

and likewise,

$$QPhiD_{i,i} = [-\Psi_i(t_1^i) - q_1^i \dot{\Psi}_i(t_1^i), -q_2^i \dot{\Psi}_i(t_2^i), \dots, -\Psi_i(t_{m_i}^i) - q_{m_i}^i \dot{\Psi}_i(t_{m_i}^i)]$$

The q^i are the quadrature weights which are determined by integration method. Notice that the first and last terms in $QPhiD_{i,i}$ have an additive boundary term which is addressed in the Python code.

Construction of y Continued

Let $X = (x_1, \dots, x_n)$ be the set of data from the n trajectories. Define Y as the y vector, as defined before expect now partially unstacked as a matrix with with d columns.

Then,

$$Y = X \cdot QPhiD^T \quad (3)$$

where $\cdot$ implies matrix-matrix multiplication. The following code constructs $QPhi$ and $QPhiD$ and then determines Y in Python.

```

# Requires a list of ts a list of qs, and outputs a block diagonal
# matrix
# corresponding to the QPhi block diagonal matrix and QPhiD block
# diagonal matrix.
# Probably the least efficient part of the code. (loops through the
# trajectories)

def Calculate_QPhi(qs,ts,n):
    psArr = []
    pspArr = []
    for i in range(len(ts)):
        ps, psp = phisp(ts[i],n,ts[i][0],ts[i][-1])
        psp = -(psp[:] * qs[i][:,None])
        # Add boundary terms to psp
        psp[0,:] = psp[0,:]-ps[0,:]
        psp[-1,:] = psp[-1,:]+ps[-1,:]
        psp = psp.T
        ps = (ps * qs[i][:,None]).T
        psArr.append(ps)
        pspArr.append(psp)
    return sp.block_diag(psArr), sp.block_diag(pspArr)

```

```

# Calculate Y, requires QPhi, QPhiD (derivatives of QPhi), data, and
# qs (to recover
# Phi from QPhi)
def Calculate_y(QPhiD,qs,dataX):
    ys = dataX@(QPhiD.T)
    return ys

```

The Matrix $M \in \mathbb{R}^{pnd \times pnd}$

M is block diagonal matrix with blocks given by

$$M_{ij} = \iint K(x_i(s), x_j(t)) ds dt \quad (4)$$

In (4) above K is a matrix-valued kernel. In the case that $K = k(x_i, x_j) \cdot I_d$ (4) becomes

$$M_{ij} = \iint k(x_i(s), x_j(t)) ds dt \otimes I_d \quad (5)$$

The double integral in M can be evaluated numerically in Python using a quadrature approximation method.

The Vectorized Guassian Kernel

A kernel function can be constructed in Python which inputs matrices $X, Y \in \mathbb{R}^{d \times \sum_i^n m_i}$ of vectors and outputs the scalar kernel value for each element of the output matrix. The following Python code accomplishes this task:

```
def gaussianK(X,Y,sigma=kernSig):  
    X2 = np.sum(X*X,axis=0)  
    Y2 = np.sum(Y*Y,axis=0)  
    XY = np.dot(X.T,Y)  
    # -X2 - Y2 + 2XY = ||X-Y||^2  
    # f(||X-Y||), f(np.sqrt(-X2 - Y2 + 2XY))  
    return np.exp((-X2[:,None]-Y2[None,:]+2*XY)/(2*sigma**2))
```

This technique of reshaping the vector 'X2','Y2' with 'None' in the index allows numpy to calculate the Gram matrix without explicitly using 'for' loops. This vectorization of the code leads to increased efficiency.

Construction of the M Matrix using Matrix Multiplication

In order to construct M we will need the Gram Matrix, G from the dataset X. The matrix $G \in \mathbb{R}^{(\sum_i^n m_i \times \sum_i^n m_i)}$ is defined as ¹

$$G = \begin{pmatrix} k(x_{1,1}, x_{1,1}) & k(x_{1,1}, x_{1,2}) & \cdots & k(x_{1,1}, x_{n,m_n}) \\ \vdots & \vdots & & \vdots \\ k(x_{n,m_n}, x_{1,1}) & k(x_{n,m_n}, x_{1,2}) & \cdots & k(x_{n,m_n}, x_{n,m_n}) \end{pmatrix}$$

Then M is given by the matrix product

$$M = QPhi \cdot (G \cdot QPhi^T) \quad (6)$$

The following Python code implements this multiplication with the Gaussian kernel.

¹we take dataX=dataY

Python Code for Constructing M

```
# QPhi is the block diagonal matrix computed by the Calculate_QPhi  
function  
def Calculate_M(QPhi,dataX,dataY,kernel=lambda X, Y : gaussianK(X,Y,  
                        sigma=kernSig)):  
    G = kernel(dataX,dataY)  
    return QPhi@(G@(QPhi.T))
```

Solving for α

We now solve for the partial-unstacked $\alpha \in \mathbb{R}^{np \times d}$ with the substitution, $Y^\top$ for y in equation (1).

```
# dataset - (dataX in  $\mathbb{R}^{d \times \sum_i m_i}$ )
# ts      - (list of time vectors for dataset (could be different
              lengths))
# qs      - (list of time vectors of quadratures (could be different
              lengths))
# n        - (number of features in the test space)
# lambdaReg - (Regularization term for the alpha)
# sigma    - (The scale parameter for the kernels)
# gaussianK - (A pointer to the implicit kernels)
def learn_alpha(dataset,ts,qs,n,lambdaReg=.0000001,sigma=kernSig,kern
                = gaussianK):
    Qphi, QphiD = Calculate_QPhi(qs,ts,n)
    M = Calculate_M(Qphi,dataset,dataset,kernel= lambda X, Y : kern(X,Y,
                          sigma=sigma))
    ys = Calculate_y(QphiD,qs,dataset)
    alphas = np.linalg.solve(M+lambdaReg*np.eye(M.shape[0]),ys.T)
    return alphas, Qphi
```

Calculation of f at New Data Points

Evaluating f along a trajectory outside the training set requires only matrix multiplications:

$$f = \alpha^T \cdot ((G \cdot (Q\Phi^T))^T) \quad (7)$$

where G and $Q\Phi$ are constructed once again with the new data points. The Python code for this calculation is below:

```
# Calculate f, requires Qhi, alpha, and the data to evaluate f at (
                                dataX) as
# well as the data of the training set (dataY). (alpha should be
                                matrix valued)

def Calculate_f(QPhi,dataX,dataY,alpha,kernel = gaussianK, sigma=
                                kernSig):
    G = kernel(dataX,dataY,sigma)
    return alpha.T@((G@(QPhi.T)).T)
```